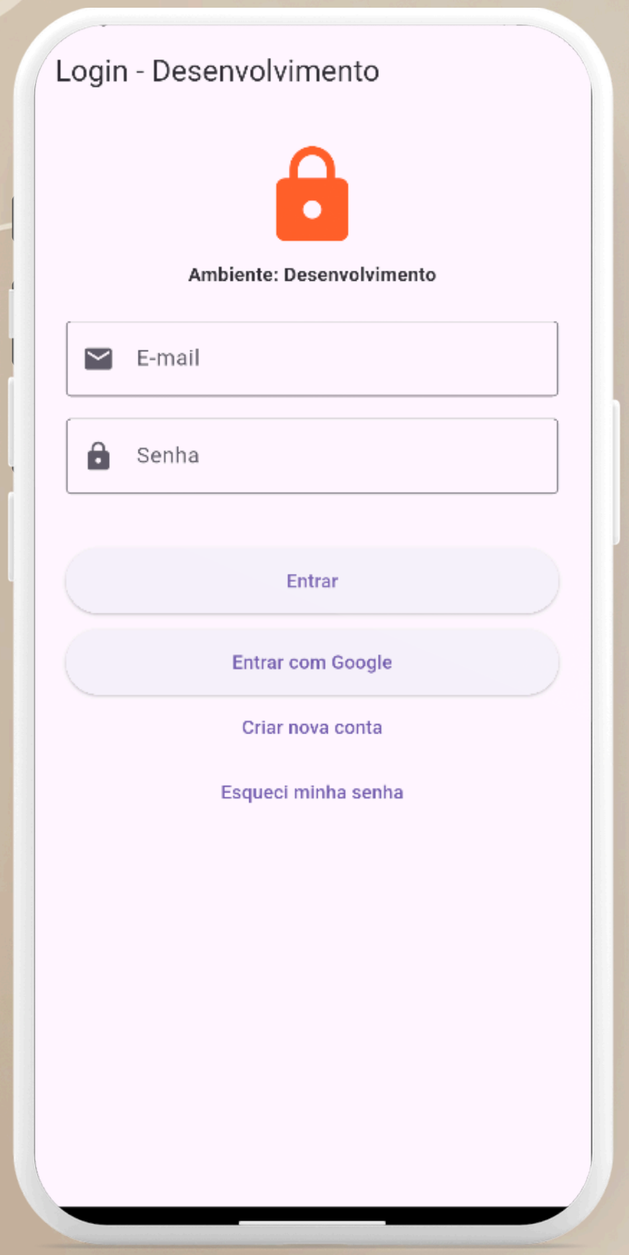


Firebase Authentication no Flutter

Cadastro, Login, Perfil no Firestore e Navegação Reativa — um guia completo do zero ao fluxo de autenticação profissional.

Integrantes do grupo:

- ***Lucca Sander Frisso***
- ***Henrique Gonçalves Pimenta Velloso***
- ***Filipe Nery Rabelo***
- ***Rafael Rehfeld***

VISÃO GERAL

O que será desenvolvido

Um fluxo completo de autenticação com Firebase, cobrindo desde o setup até a navegação reativa.

Tecnologias

- Flutter
- Firebase Authentication
- Cloud Firestore
- FlutterFire CLI

Funcionalidades

- Cadastro e login com e-mail/senha
- Login com Google
- Recuperação e verificação de e-mail
- Persistência automática de sessão
- Perfil do usuário no Firestore
- Regras de segurança e navegação reativa

ARQUITETURA

Fluxo da Aplicação

A arquitetura separa responsabilidades em camadas distintas, garantindo código testável e escalável.

Flutter App

Camada de UI e interação

Firebase Auth

Gerencia identidade do usuário

Cloud Firestore

Camada de dados em tempo real

/users/{uid}

Coleção de perfis e armazenamento

O **Firebase Authentication** gerencia identidade, o **Firestore** armazena perfis, os **Streams** atualizam telas automaticamente e as **Rules** controlam acesso aos dados.

CONFIGURAÇÃO

Criando o Projeto Firebase

01

Acessar o Firebase Console

console.firebase.google.com → criar novo projeto

02

Ativar os Serviços

Authentication (Email/Password + Google) e Cloud Firestore

03

Registrar os Apps

Android, iOS e Web — baixar os arquivos de configuração

04

Habilitar Provedores

Email/Password e Google Sign-In na seção Authentication

CONFIGURAÇÃO

FlutterFire CLI e Dependências

Instalar e Configurar

```
dart pub global activate flutterfire_cli
flutterfire configure
```

Gera automaticamente lib/firebase_options.dart com API Keys e IDs do projeto.

pubspec.yaml

```yaml
dependencies:
  firebase_core: ^3.0.0
  firebase_auth: ^5.0.0
  cloud_firestore: ^5.0.0
  google_sign_in: ^6.0.0
```

```
flutter pub get
```

CÓDIGO

Inicializando o Firebase

main.dart

```
void main() async {
  WidgetsFlutterBinding
    .ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions
      .currentPlatform,
  );

  runApp(const MyApp());
}
```

Importe firebase_core.dart e firebase_options.dart. A inicialização deve ocorrer **antes** de runApp().

Estrutura Recomendada

services/
Firebase Auth

repositories/
Firestore

screens/
UI

models/
Dados

CÓDIGO

Serviço de Autenticação

```
class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Stream<User?> get authState => _auth.authStateChanges();

  Future<UserCredential> signUp(String email, String password) {
    return _auth.createUserWithEmailAndPassword(
      email: email, password: password);
  }

  Future<UserCredential> signIn(String email, String password) {
    return _auth.signInWithEmailAndPassword(
      email: email, password: password);
  }

  Future<void> signOut() => _auth.signOut();
}
```

Centralizar a lógica em um AuthService facilita manutenção, testes e reuso em toda a aplicação.

CÓDIGO

Cadastro e Login

Cadastro de usuário

```
Future<UserCredential> signUp({
  required String email,
  required String password,
}) async {
  return await FirebaseAuth.instance
      .createUserWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

Login

```
Future<UserCredential> login({
  required String email,
  required String password,
}) async {
  return await FirebaseAuth.instance
      .signInWithEmailAndPassword(
    email: email,
    password: password,
  );
}
```

SEGURANÇA

Regras de Segurança no Firestore

As regras garantem que apenas o próprio usuário acesse seus dados.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write:
        if request.auth != null &&
           request.auth.uid == userId;
    }
  }
}
```

O que as regras fazem

- Verificam se o usuário está autenticado
- Permitem acesso apenas ao próprio documento
- Bloqueiam leitura/escrita não autorizada

NAVEGAÇÃO

Navegação Reativa e Próximos Passos

```
StreamBuilder<User?>(
  stream: AuthService().authState,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.active) {
      return snapshot.hasData
        ? const HomeScreen()
        : const LoginScreen();
    }
    return const SplashScreen();
  },
)
```

O StreamBuilder ouve mudanças no estado de autenticação e alterna automaticamente entre LoginScreen, HomeScreen e SplashScreen.

BOAS PRÁTICAS

Tratamento de Erros da API

Firebase lança FirebaseAuthException com codes específicos — sempre capture e exiba mensagens amigáveis.

```
try {
  await AuthService().signIn(email, password);
} on FirebaseAuthException catch (e) {
  switch (e.code) {
    case 'user-not-found':
      showError('E-mail não cadastrado.');
      break;
    case 'wrong-password':
      showError('Senha incorreta.');
      break;
    case 'email-already-in-use':
      showError('E-mail já utilizado.');
      break;
    case 'weak-password':
      showError('Senha fraca — mínimo 6 caracteres.');
      break;
    default:
      showError('Erro: ${e.message}');
  }
}
```

Principais error codes

`user-not-found`
E-mail não cadastrado

`wrong-password`
Senha incorreta

`email-already-in-use`
E-mail já em uso

`weak-password`
Senha muito fraca

`too-many-requests`
Muitas tentativas

`network-request-failed`
Sem conexão

Verificação e Redefinição de E-mail

Enviar verificação

```
final user = FirebaseAuth
  .instance.currentUser;

if (user != null &&
    !user.emailVerified) {
  await user
    .sendEmailVerification();
  // Avise o usuário!
}

// Checagem posterior:
await user?.reload();
final verified =
  user?.emailVerified ?? false;
```

Redefinição de senha

```
  await FirebaseAuth
    .instance
    .sendPasswordResetEmail(
      email: emailController.text
        .trim(),
    );

  // Firebase envia um link
  // automático. Não é necessário
  // implementar a troca de senha
  // manualmente.
```

⚠ **Boa prática:** bloqueie o acesso a rotas protegidas enquanto emailVerified == false. Cheque sempre após reload().

Login com Google

Pré-requisitos

1. Habilitar Google Sign-In no Firebase Console
2. Adicionar SHA-1 e SHA-256 nas configurações Android
3. Adicionar google_sign_in ao pubspec.yaml
4. Baixar novo google-services.json após adicionar SHA

Obter SHA no Android

```
cd android && ./gradlew signingReport
```

Implementação

```
Future<UserCredential?> signInWithGoogle() async {
  final googleUser =
    await GoogleSignIn().signIn();
  if (googleUser == null) return null;

  final googleAuth =
    await googleUser.authentication;

  final credential =
    GoogleAuthProvider.credential(
      accessToken: googleAuth.accessToken,
      idToken:     googleAuth.idToken,
    );

  return await FirebaseAuth.instance
    .signInWithCredential(credential);
}
```

Criando o Perfil no Firestore

Logo após o cadastro, grave um documento em /users/{uid} com os dados iniciais do usuário.

```
Future<void> createUserProfile(UserCredential credential) async {
  final user = credential.user!;

  await FirebaseFirestore.instance
    .collection('users')
    .doc(user.uid)
    .set({
      'uid':        user.uid,
      'name':       user.displayName ?? '',
      'email':      user.email ?? '',
      'photoUrl':   user.photoURL ?? '',
      'role':       'user',          // autorização simples
      'createdAt':  FieldValue.serverTimestamp(),
      'updatedAt':  FieldValue.serverTimestamp(),
    });
}

// Chamar imediatamente após signUp / signInWithGoogle:
final credential = await authService.signUp(email, password);
await createUserProfile(credential);
```

Estrutura do documento

uid	string
name	string
email	string
photoUrl	string
role	string
createdAt	timestamp
updatedAt	timestamp

Armazene apenas o necessário — respeite a LGPD.

Leitura e Atualização de Perfil

Leitura em tempo real (stream)

```
Stream<DocumentSnapshot> userStream(String uid) {
  return FirebaseFirestore.instance
    .collection('users')
    .doc(uid)
    .snapshots();
}

// Na UI:
StreamBuilder<DocumentSnapshot>(
  stream: userStream(currentUser.uid),
  builder: (context, snapshot) {
    final data = snapshot.data?.data()
      as Map<String, dynamic>?;
    final name = data?['name'] ?? '';
    return Text(name);
  },
)
```

Atualização de perfil

```
Future<void> updateProfile({
  required String uid,
  required String newName,
  String? newPhotoUrl,
}) async {
  await FirebaseFirestore.instance
    .collection('users')
    .doc(uid)
    .update({
      'name':      newName,
      if (newPhotoUrl != null)
        'photoUrl': newPhotoUrl,
      'updatedAt': FieldValue
        .serverTimestamp(),
    });
}
```

💡 Prefira .update() ao invés de .set() para preservar campos existentes que você não está alterando.

SEGURANÇA

Regras por UID e Role

Controle quem lê e quem escreve usando campos como role no próprio documento do usuário.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Perfil: só o próprio usuário lê/escreve
    match /users/{userId} {
      allow read, write:
        if request.auth != null &&
           request.auth.uid == userId;
    }

    // Recurso protegido: só admins lêem
    match /admin-data/{doc} {
      allow read:
        if request.auth != null &&
           get(/databases/$(database)/documents/
               users/$(request.auth.uid)
           ).data.role == 'admin';
      allow write: if false; // nunca pelo cliente
    }

    // Posts: autenticado cria, todos lêem
    match /posts/{postId} {
      allow read: if true;
      allow create:
        if request.auth != null &&
           request.resource.data.authorId
             == request.auth.uid;
      allow update, delete:
        if request.auth.uid ==
           resource.data.authorId;
    }
  }
}
```

Acesso por UID

Cada usuário só toca seu próprio documento /users/{uid}

Role-based access

get() lê o papel do usuário em tempo real nas regras

Nunca confie no cliente

allow write: if false bloqueia escrita direta de dados sensíveis

Teste no Emulator

firebase emulators:start valida as regras sem custo

BOAS PRÁTICAS

Separação de Ambientes Dev / Prod

Desenvolvimento

- Projeto Firebase separado (myapp-dev)
- google-services.json de dev em android/app
- FLUTTER_APP_ENV=development no launch.json
- Firestore Emulator para testes locais
- Regras permissivas (allow read, write: if true)

Produção

- Projeto Firebase separado (myapp-prod)
- google-services.json de prod via CI/CD
- Chaves nunca no repositório (use .env / secrets)
- Regras rígidas de segurança validadas
- Monitoring e alertas ativados

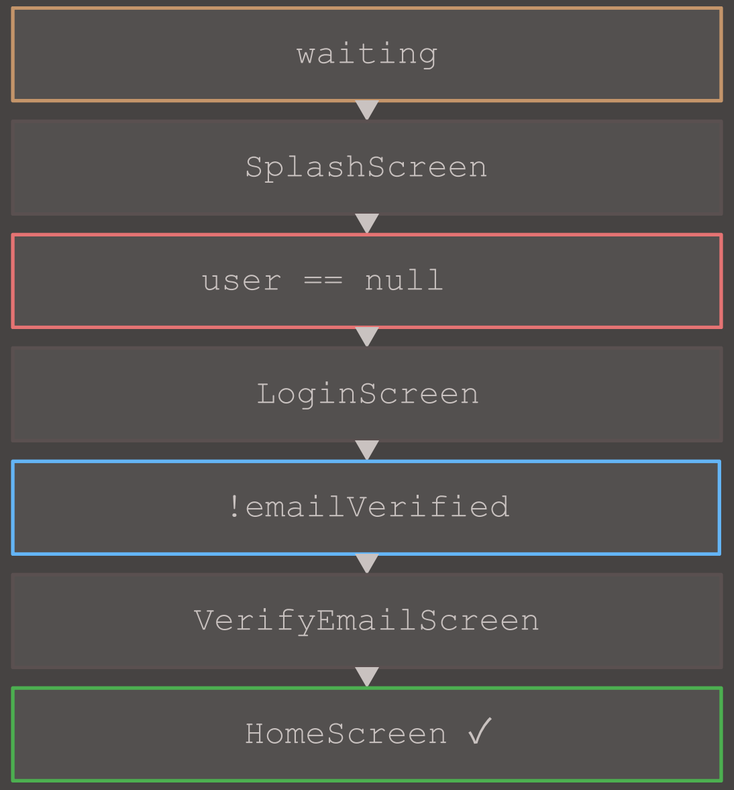
```
// main.dart — trocar firebase_options conforme ambiente
const env = String.fromEnvironment('FLUTTER_ENV', defaultValue: 'dev');
```

Persistência de Sessão

O Firebase persiste a sessão automaticamente. O stream authStateChanges() é a fonte de verdade do estado da autenticação.

```
// app_router.dart — navegação reativa completa
Widget build(BuildContext context) {
  return StreamBuilder<User?>(
    stream: FirebaseAuth.instance.authStateChanges(),
    builder: (context, snapshot) {

      // Aguardando a resposta inicial do Firebase
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const SplashScreen();
      }

      final user = snapshot.data;

      if (user == null) {
        return const LoginScreen();           // Não autenticado
      }

      if (!user.emailVerified) {
        return const VerifyEmailScreen();     // Aguardando verificação
      }

      return const HomeScreen();             // Autenticado ✓
    },
  );
}
```

Fluxo de estados



PRIVACIDADE

Privacidade e Boas Práticas

Mínimo de dados

Colete apenas o necessário (LGPD/GDPR). Não grave CPF, telefone ou endereço sem consentimento explícito.

Teste em dispositivo real

Google Sign-In e fluxos OAuth falham em emuladores sem conta configurada. Sempre valide no device físico.

Nunca exponha API Keys no código

As chaves do Firebase são públicas por design — proteja com regras de segurança, não ocultando a chave.

Exclusão de conta

Ofereça ao usuário a opção de deletar conta (exige reautenticação recente) e remova os dados do Firestore.

SHA-1 e SHA-256 no Android

Necessários para Google Sign-In. Adicione tanto o keystore de debug quanto o de release no console Firebase.

Política de privacidade

Obrigatória para apps com login social. Requerida pelas lojas App Store e Google Play.

CONCLUSÃO

Checklist do Fluxo Completo

- ✓ Projeto Firebase criado e serviços ativados (Auth + Firestore)
- ✓ FlutterFire CLI configurado — firebase_options.dart gerado
- ✓ Fluxo e-mail/senha: cadastro, login e logout funcionando
- ✓ Google Sign-In com SHA-1/SHA-256 configurados
- ✓ Verificação de e-mail e redefinição de senha implementadas
- ✓ Tratamento de erros com FirebaseAuthException
- ✓ Perfil criado em /users/{uid} após cadastro (nome, foto, role, datas)
- ✓ Leitura e atualização de perfil via Firestore stream
- ✓ Regras de segurança por UID e role no Firestore
- ✓ Navegação reativa com StreamBuilder + authStateChanges()
- ✓ Ambientes dev/prod separados com projetos Firebase distintos
- ✓ Privacidade: apenas dados necessários, política de privacidade

Autenticação profissional: segura, reativa, com perfil no Firestore e separação de ambientes!

Desafio — Planejar um Fluxo de Autenticação

Em grupos, vocês devem desenhar ou estruturar como seria um sistema de autenticação utilizando Firebase

O fluxo deve conter:

- tela de login;
- tela de cadastro;
- recuperação de senha;
- armazenamento do perfil do usuário;
- caminho que os dados percorrem entre Flutter, Firebase Auth e Firestore.”

Devem ser entregue algum desses itens:

- desenho no papel;
- fluxograma;
- slide;
- Figma;
- ou até pseudocódigo simples.

Gabarito

- Usuário abre o app
- App verifica se já existe sessão ativa
- Se não tiver sessão, mostra a tela de login
- Usuário pode escolher:
- fazer login com e-mail e senha;
- criar uma nova conta;
- recuperar senha.
- No cadastro, o Firebase Authentication cria o usuário
- O app pega o uid desse usuário
- O Firestore salva os dados em: users/{uid}
- Após login/cadastro, o usuário vai para a tela principal
- Se fizer logout, volta para a tela de login

O Firebase Authentication cuida da identidade e do login do usuário, enquanto o Cloud Firestore armazena os dados extras do perfil, usando o uid como identificador único.

Obrigado!

Material de apoio: https://github.com/Filnr/Seminario-Firebase